

RAILWAY MANAGEMENT SYSTEM

Complete Setup Guide, Architecture & Professor Viva Presentation Manual

PROJECT LEAD & DEVELOPER

VIKAS YADAV

Database Management Systems (DBMS) Capstone Project

RELATIONAL DATABASE ENGINE
PostgreSQL 16 (ACID Engine)

APPLICATION FRAMEWORK
Next.js 15 + TypeScript

STYLING & UI COMPONENTS
Tailwind CSS + Lucide Icons

GITHUB REPOSITORY
Lanthanode/PROJECT-VIKAS

Department of Computer Science & Engineering

Real-Life Case Study: Indian Railways Passenger Reservation System (IRCTC)

Includes One-Click Batch Launcher • Zero External DB Setup • Live PL/pgSQL Triggers • Automated Auditing

1. Quick Start & Execution Options

The Railway Management System is built to be **100% self-contained**. It embeds an official PostgreSQL 16 WebAssembly engine (`@electric-sql/pglite`), eliminating the need for examiners or students to manually install, configure, or start a PostgreSQL background daemon.

One-Click Windows Launcher: For instant presentation to college professors, double-click `START_RAILWAY_SYSTEM.bat` in the project folder. It verifies Node.js, installs any missing packages, starts the server, and automatically launches your browser to `http://localhost:3000`.

Execution Methods Summary

Method	Command / Action	Environment	Best For
One-Click Batch	Double-click <code>START_RAILWAY_SYSTEM.bat</code>	Windows Local	College presentation & viva demonstration
Dev Server	<code>npm run dev</code>	Cross-platform	Interactive development & live debugging
Production Build	<code>npm run build && npm run start</code>	Cross-platform	High-speed compiled production showcase
Remote PostgreSQL	Set <code>DATABASE_URL</code> in <code>.env.local</code>	Cloud / Supabase / Neon	Hosted cloud deployment

2. System Architecture Layers

The system adheres to strict software engineering separation of concerns across 5 distinct layers:

Layer	Path / Module	Responsibilities
Interface Layer	<code>/app</code> , <code>/components</code>	Responsive UI, booking modal, boarding pass, dark green branding, mobile viewport adaptability.
Integration Layer	<code>/app/api/...</code>	REST API endpoints for trains, stations, bookings, PNR lookups, and SQL query execution.
Logic Layer	<code>/lib/services/railway.ts</code>	Seat assignment algorithms, duplicate prevention validation, fare calculations, PNR formatting.
Storage Layer	<code>/lib/db.ts</code> , <code>/database/</code>	PostgreSQL 16 relational engine, DDL schema, views, triggers, and ACID transactions.
Security Layer	<code>/app/admin</code>	Master administrative authentication gate (password: <code>admin123</code>) and audit logging.

3. Step-by-Step Installation Guide

1 Clone the GitHub Repository

Clone the project onto your local workstation using Git:

```
git clone https://github.com/Lanthanode/PROJECT-VIKAS.git
cd PROJECT-VIKAS
```

2 Verify Node.js Version

Ensure Node.js 18+ or 20+ is installed on your computer:

```
node -v
npm -v
```

If not installed, download the official LTS release from <https://nodejs.org/>.

3 Install Project Dependencies

Install all required frontend and database packages:

```
npm install
```

4 Automatic Database Engine Initialization

No manual `psql` or database creation commands are needed! When you start the app, `/lib/db.ts` executes the complete DDL schema, compiles all views, installs the PL/pgSQL audit trigger, and inserts all PDF sample records automatically.

Optional: To connect to an external PostgreSQL instance (e.g. Supabase, Neon), create `.env.local`:

```
DATABASE_URL="postgresql://postgres:[PASSWORD]@[HOST]:5432/[DB_NAME]?sslmode=require"
```

5 Launch the Application

```
npm run dev
```

Open your browser and navigate to: `http://localhost:3000`

4. Core Application URL Reference

Page / Feature	URL Route	Primary Purpose
Homepage / Dashboard	<code>http://localhost:3000/</code>	Hero ticket, live stats bar, available trains, major stations, recent bookings.

Page / Feature	URL Route	Primary Purpose
Train Schedule	http://localhost:3000/trains	Search by origin/destination stations, view departure/arrival timings and base fares.
Network Stations	http://localhost:3000/stations	Station catalog with station codes (NDLS, BCT, CDG, PUNE) and connected trains.
Bookings & Cancellation	http://localhost:3000/bookings	Full table of reservations with instant seat cancellation and trigger logging.
PNR Status & Ticket	http://localhost:3000/pnr	Search any PNR number to display a printable executive digital boarding pass.
DBMS Lab & SQL Runner	http://localhost:3000/dbms-lab	Live SQL query runner, views, and PL/pgSQL trigger audit trail inspector.
Admin Portal (CRUD)	http://localhost:3000/admin	Protected portal for inserting and managing Passengers, Trains, and Stations.
About & Team Details	http://localhost:3000/about	System specifications, workflow diagram, and academic viva reference notes.

5. College Professor & Viva Presentation Script

Follow this exact 10-step sequence during your project demonstration to prove relational database concepts to the examiner:

1 Show Live Database Dashboard Statistics

Open `http://localhost:3000`. Point out the dark green statistics bar showing Active Trains (5), Stations (6), Passengers (9), and Confirmed Bookings. Explain that these figures are dynamically computed from PostgreSQL aggregate queries (`COUNT(*)` , `SUM(Amount)`).

2 Navigate to Train Schedule

Go to `/trains`. Search "Delhi" to "Mumbai". Show **Rajdhani Express (Train 1001)** departing at 16:30 and arriving at 08:00, exactly matching the capstone PDF.

3 Execute Live Ticket Booking

Click "Book Ticket". Enter **Rahul Sharma**, Age **21**, Gender **Male**, Phone **9876543210**. Choose Train 1001 and select UPI payment. Click "Confirm Reservation". Show the celebratory confetti and instant PNR generation.

4 Demonstrate Duplicate Seat Prevention Constraint

Attempt to book the exact same seat on Train 1001 for the same journey date. Show the professor that the system returns an error: *"Seat is already occupied on train 1001"*. Explain that this is enforced by `CONSTRAINT unique_train_journey_seat UNIQUE (Train_ID, Journey_Date, Seat_Number)` in PostgreSQL.

5 Verify Automated Database Trigger Audit

Navigate to `/dbms-lab` → click **"Trigger Audit Trail"**. Show the newly inserted record in `Reservation_Audit` containing `RESERVATION_CREATED`, timestamp, and reservation details automatically populated by `trg_reservation_audit` without any manual logging code.

6 Display Printable Boarding Pass & PNR Status

Navigate to `/pnr` and enter the generated PNR number. Display the digital boarding pass showing Passenger details, Train number, Seat, Departure, and Fare.

7 Demonstrate Ticket Cancellation & Seat Release

Click "Cancel Ticket". Observe that the status immediately transitions from `Confirmed` to `Cancelled`, freeing up the seat in the inventory and generating a `RESERVATION_CANCELLED` audit row.

8 Execute Capstone SQL Queries in DBMS Lab

In `/dbms-lab`, run:

- **Simple Query:** `SELECT * FROM Passenger WHERE Age > 20;`
- **Nested Subquery:** `SELECT Name FROM Passenger WHERE Passenger_ID IN (SELECT Passenger_ID FROM Reservation);`
- **Two-Table JOIN:** Combines Passenger and Reservation tables across foreign keys.
- **Relational View:** `SELECT * FROM Passenger_Reservation_View;`

9 Open Protected Admin Portal (CRUD)

Go to `/admin`, authenticate with `admin123`, and show live CRUD operations on Passengers, Trains, and Stations.

10 Summarize DBMS Architecture

Conclude by explaining why relational normalization matters, why foreign keys prevent orphan rows, and how the bridge table `Train_Station` handles Many-to-Many routing.

6. Troubleshooting & Frequently Asked Questions

Symptom / Question	Root Cause	Resolution
Port 3000 is occupied	Another service or zombie Node process is running on port 3000.	Run on port 3005: <code>npx next dev -p 3005</code> or terminate the process using <code>npx kill-port 3000</code> .
'node' is not recognized	Node.js is not installed or not added to system <code>PATH</code> .	Install Node.js 18+ from https://nodejs.org/ and restart your terminal.
Database tables missing	First-time initialization was interrupted.	Delete the <code>/data</code> folder and re-launch the application. The system will recreate all tables, triggers, and seed data.
Can I connect to Supabase or Neon?	Yes, standard PostgreSQL connection strings are fully supported.	Set <code>DATABASE_URL="postgresql://user:pass@host:5432/db"</code> in <code>.env.local</code> .
Is a payment gateway required?	No, this is an academic capstone.	Payments are recorded realistically in the <code>Payment</code> SQL table without external credit card processing.

7. Verification & Quality Checklist

- ✓ **PostgreSQL 16 Engine:** Active and fully ACID compliant.
- ✓ **Primary & Foreign Keys:** Strictly enforced across all 6 tables.
- ✓ **Duplicate Seat Prevention:** Enforced by composite UNIQUE constraint.
- ✓ **PL/pgSQL Trigger:** Automatically logs INSERT and UPDATE actions to `Reservation_Audit`.
- ✓ **Compiled Views:** `Passenger_Reservation_View` queries successfully.
- ✓ **One-Click Launcher:** `START_RAILWAY_SYSTEM.bat` launches seamlessly.
- ✓ **Responsive Design:** Tested across 375px, 768px, 1024px, and 1440px viewports.